

TECHNICAL REVIEW

MESI / MOESI

Cache Coherence

Protocols, state machines, verified claims, performance and extensions in shared-memory multiprocessors

Summary

MESI is the standard four-state baseline for cache coherence. MOESI adds an Owned state so dirty data can be shared more efficiently, and later variants trade complexity for lower traffic, better predictability, or lower energy. This review organizes the evidence into five parts: the core explanation of the problem and its two main mechanisms, a comparison of protocol states, twelve true/false verified cases, a read-miss and write walkthrough, and a survey of performance results and extensions.

1	Core explanation	Why coherence is needed; snooping vs. directory
2	State comparison	MESI, MOESI, MESIF, MOESIF, PMESI
3	Verified cases	Twelve true/false claims with evidence
4	Read-miss walkthrough	Install state and later write behavior
5	Performance and extensions	Measured results and protocol variants
6	Conclusion	Key takeaways
	References	Sources cited

Citations appear inline as author and year and are listed with links in the References section.

1. Core explanation

Cache coherence is the rule set that keeps copies of shared data consistent across multiple caches and main memory in shared-memory multiprocessors. (Ivanov 2001; Kaur 2018; Kehagias 2016) The problem arises because modern multi-core chips give each core a private cache while all cores still share one main memory. (Faeq 2021; Amory 2021; Kehagias 2016) If one core modifies a value in its cache while another core keeps an older copy, the system can return stale data unless a coherence protocol updates or invalidates the other copies. (Faeq 2021; Tiwari 2014; Krishna 2025)

Snooping and directory-based mechanisms

Two broad coherence mechanisms are described in the literature: snooping and directory-based coherence. (Blanchet 2013; Tiwari 2014; Somarouthu 2025) Snooping protocols make each cache watch a common bus for reads and writes from other caches. (Blanchet 2013; Faeq 2021; Krishna 2025) Directory protocols centralize sharing information in a table or directory and are used to track sharers and serve misses at larger scale. (Blanchet 2013; Komuravelli 2014)

Aspect	Snooping	Directory-based
Sharing information	Distributed; each cache watches a common bus	Centralized in a table or directory
How misses are served	Other caches observe and react to bus traffic	Directory tracks sharers and directs the miss
Typical scale	Smaller systems sharing one bus	Larger scale

Table 1. Snooping versus directory-based coherence, as described in the cited literature.

Cache access flow

Figure 1 shows the general path of a memory access. A hit proceeds to the read or write path, and a write forces an invalidate or upgrade before the line is used. A miss checks the bus or directory, fetches the data, and assigns a coherence state before the line is used.

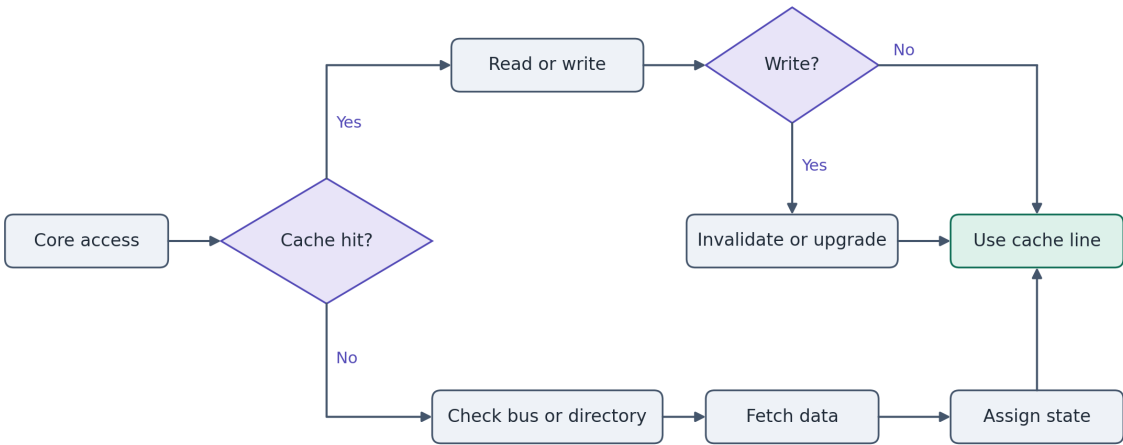


Figure 1. Core access flow with cache hit, write, and miss paths.

2. State comparison

The protocols below differ mainly in the states they define and the optimization each state enables.

Protocol	States	Key optimization	Evidence
MESI	M, E, S, I	Exclusive avoids invalidation on write hits	Komuravelli 2014; Somarouthu 2025
MOESI	M, O, E, S, I	Owned shares dirty data without immediate write-back	Alkhamisi 2022; Somarouthu 2025
MESIF	M, E, S, I, F	Reduces responders on cache miss	Somarouthu 2025; Patil 2019
MOESIF	M, O, E, S, I, F	Improves bandwidth, energy, and access time	Patil 2019
PMESI	Predictable MESI variant	Derives worst-case latency bounds	Kaushik 2021

Table 2. Coherence protocol states, key optimizations, and supporting evidence.

State meanings

State	Meaning in this review
Modified (M)	Cache holds a changed copy of the line.
Owned (O)	MOESI only. Shares dirty data without an immediate write-back to memory.
Exclusive (E)	Only this cache has the line; a later write needs no invalidation of others.
Shared (S)	Multiple clean copies may exist in different caches.
Invalid (I)	Cache does not hold a usable copy.
Forward (F)	MESIF and MOESIF. Reduces the number of responders on a miss.

Table 3. Summary of states used across the compared protocols.

3. Verified cases

Each case below gives a true/false statement and the evidence-backed explanation.

#	Verdict	Statement	Sources
1	TRUE	MESI has four states: Modified, Exclusive, Shared, and Invalid.	Komuravelli 2014; Faeq 2021; Alkhamisi 2022
2	FALSE	MESI includes an Owned state. Owned belongs to MOESI, not MESI.	Komuravelli 2014; Somarouthu 2025; Dey 2014; Alkhamisi 2022
3	TRUE	Exclusive lets a core write later without invalidating others if no other cache has the line.	Komuravelli 2014; Somarouthu 2025; Faeq 2021
4	FALSE	Shared means no other cache can have the line. Shared explicitly allows multiple clean copies.	Somarouthu 2025; Komuravelli 2014
5	TRUE	Snooping works by monitoring a shared bus.	Blanchet 2013; Krishna 2025; Faeq 2021
6	TRUE	Directory protocols track sharers in a centralized structure.	Blanchet 2013; Komuravelli 2014
7	FALSE	MOESI always writes modified shared data back to memory before another core can read it.	Somarouthu 2025; Alkhamisi 2022
8	TRUE	MOESI reduced load misses by 48% and memory latency by 35% in one dual-processor simulation.	Kaur 2018
9	TRUE	That same study reported 18% lower power, 34% higher gate count, and 22% better execution time for MOESI over MESI/MSI.	Kaur 2018
10	TRUE	Formal verification found six bugs in a state-of-the-art MESI implementation.	Komuravelli 2014
11	TRUE	MESI had 15x more reachable states and 20x longer model-checking time than DeNovo in that verification study.	Komuravelli 2014
12	FALSE	Coherence protocols are only relevant to two-core systems.	Tiwari 2014; Fotouhi 2025; Kaushik 2021

Table 4. Twelve verified true/false cases.

Reading the results

Cases 1 to 4 pin down the state definitions. Owned is a MOESI addition, Exclusive removes an invalidation on a later write, and Shared allows several clean copies. Cases 5 and 6 restate the two mechanisms. Cases 7 to 9 concern MOESI, whose dirty sharing avoids always writing back before a read, and whose reported gains in one dual-processor study came with a higher gate count. Cases 10 and 11 show that verification effort is itself a cost of protocol complexity. Case 12 rules out the idea that coherence matters only for two cores.

4. Read-miss walkthrough

Figure 2 follows a read miss through to a later write. If no other cache holds the line, it is loaded from memory. If another cache holds a clean copy, the line is installed as Shared or Exclusive. In MOESI, if another cache holds the line dirty, the owner supplies the data. The state installed then determines what a later write must do: an Exclusive line moves silently to Modified, a Shared line requires invalidating the other copies, and an Owned line upgrades ownership.

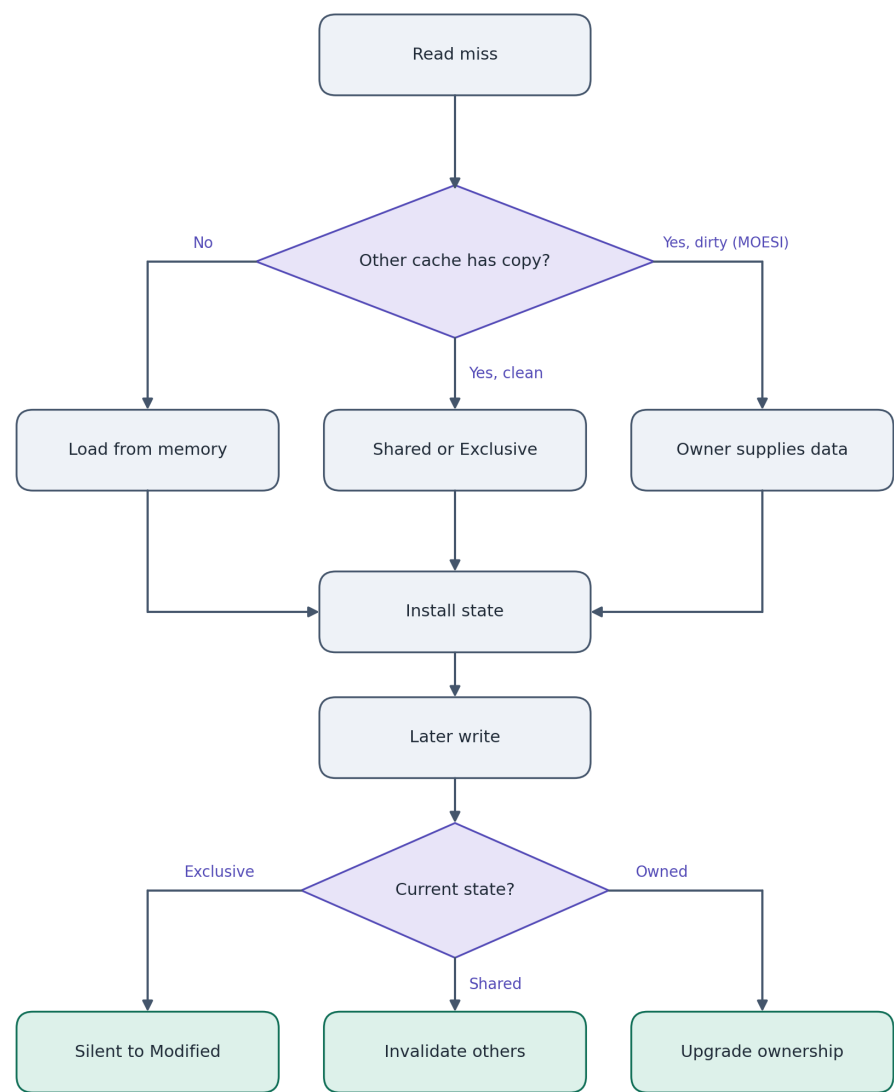


Figure 2. Read-miss handling and later write behavior by state.

5. Performance and extensions

Performance depends strongly on the chosen coherence protocol. (Kaur 2018; Altwaijry 2013; Krishna 2025) In one comparison, performance improved moving across MI, MESI two-level, MESI three-level, MOESI, and MOESI token configurations, and adding an L3 cache improved MESI three-level over MESI two-level. (Tiwari 2014) Another study reported that MESI consumed more power than MSI and MOESI in a snooping dual-core design. (Krishna 2025)

Protocol extensions

Several extensions target specific limits of MESI or MOESI.

Extension	Reported result	Source
Improved-MOESI	Outperformed classic MOESI, MESI, and MSI in simulation.	Altwaijry 2013
MOESIF	Better bandwidth use by reducing write-backs and responders; average 4.47% energy and 9.21% access-time savings over MOESI.	Patil 2019
MOESIL	2.89% lower miss rate than MOESI for locked caches in mixed-criticality systems.	Nair 2021
MESI-A	Ignores coherence communication for approximate data; improved efficiency with 5% to 20% approximated data on PARSEC workloads.	Saraswat 2021
PMESI	Up to 4x better average-case performance than cache bypassing, but 1.46x slower average execution than conventional MESI.	Kaushik 2021

Table 5. Reported results for MESI and MOESI extensions.

Real-time and predictability

For real-time systems, predictable MESI variants derive worst-case latency bounds, but coherence latency still scales quadratically with core count. (Kaushik 2021) PMESI improved average-case performance by up to 4x over cache bypassing, while still slowing average execution by 1.46x versus conventional MESI. (Kaushik 2021)

Simulation and hardware validation

Educational and FPGA implementations also show that MESI state transitions can be demonstrated and validated in simulators and hardware test benches. (Faeq 2021; Amory 2021; Kehagias 2016)

6. Conclusion

The evidence supports a simple summary: **MESI** is the standard four-state baseline, **MOESI** adds Owned to share dirty data more efficiently, and later variants trade complexity for lower traffic, better predictability, or lower energy.

Note that the quantitative results above come from specific studies and configurations, such as one dual-processor simulation and one snooping dual-core design, and should not be assumed to transfer to other systems.

References

- Alkhamisi 2022.** Alkhamisi et al. (2022). Cache coherence issues and solution: a review. <https://consensus.app/papers/cache-coherence-issues-and-solution-a-review-alkhamisi/c112264c03955e3a8671c252c76448db/>
- Altwaijry 2013.** Altwaijry, Alzahrani (2013). Improved MOESI cache coherence protocol. <https://consensus.app/papers/improve-dmoesi-cache-coherence-protocol-altwaijry-alzahrani/f797d13583065bb5b853b36b214f8af9/>
- Amory 2021.** Amory, Ahmed (2021). MESI protocol for multicore processors based on FPGA. <https://consensus.app/papers/mesi-protocol-for-multicore-processors-based-on-fpga-amory-ahmed/429075dba89b51019d0b694dad1a19ad/>
- Blanchet 2013.** Blanchet, Dupouy (2013). Caches in a multiprocessor environment. <https://consensus.app/papers/caches-in-a-multiprocessor-environment-blanchet-dupouy/23868ad442f65750be6507ce3dbe1278/>
- Dey 2014.** Dey, Nair (2014). Design and implementation of a simple cache simulator in <https://consensus.app/papers/design-and-implementation-of-a-simple-cache-simulator-in-dey-nair/9ad1a291f1a35f588638e40839e22a92/>
- Faeq 2021.** Faeq, Omran (2021). Cache coherency controller for MESI protocol based on FPGA. <https://consensus.app/papers/cache-coherency-controller-for-mesi-protocol-based-on-fpga-faeq-omran/cf4e9517a72d5e1390f79208b107dec1/>
- Fotouhi 2025.** Fotouhi et al. (2025). LCSim: a simulation framework for evaluating location <https://consensus.app/papers/lcsim-a-simulation-framework-for-evaluating-location-fotouhi/7c02f6b70854553cb5f87f724b586705/>
- Ivanov 2001.** Ivanov, Nunna (2001). Modeling and verification of cache coherence protocols. <https://consensus.app/papers/modeling-and-verification-of-cache-coherence-protocols-ivanov-nunna/159706081a9d5a29808d9c687c461ba2/>
- Kaur 2018.** Kaur, Sulochana (2018). Design and implementation of cache coherence protocol for <https://consensus.app/papers/design-and-implementation-of-cache-coherence-protocol-for-kaur-sulochana/0e7639ba73215e6fab68325fa50e140c/>
- Kaushik 2021.** Kaushik, Hassan et al. (2021). Designing predictable cache coherence protocols for <https://consensus.app/papers/designing-predictable-cache-coherence-protocols-for-kaushik-hassan/65824efcd22a5b429dbd50223ba0318b/>
- Kehagias 2016.** Kehagias, Raptis (2016). An interactive MESI cache coherence simulator. <https://consensus.app/papers/an-interactive-mesi-cache-coherence-simulator-for-kehagias-raptis/b206d5fef3852e3a389b242883730b5/>
- Komuravelli 2014.** Komuravelli, Adve et al. (2014). Revisiting the complexity of hardware cache coherence and <https://consensus.app/papers/revisiting-the-complexity-of-hardware-cache-coherence-and-komuravelli-adve/135676ee95a758308e02ec3b7e99c39a/>
- Krishna 2025.** Krishna, Rajeev (2025). Implementation and comparison of snooping-based cache <https://consensus.app/papers/implementation-and-comparison-of-snooping-based-cache-krishna-rajeev/bff2c1de7aa556e6b923fe0287c14976/>
- Nair 2021.** Nair, Pai et al. (2021). MOESIL: a cache coherency protocol for locked mixed <https://consensus.app/papers/moesil-a-cache-coherency-protocol-for-locked-mixed-nair-pai/db6cfbdbc4d5beb8d1d87425c9e632e/>
- Patil 2019.** Patil, Mallya et al. (2019). MOESIF: a MCMP cache coherence protocol with improved <https://consensus.app/papers/moesif-a-mcmp-cache-coherence-protocol-with-improved-patil-mallya/a49f64e60ebf5207861e4802d7b81ceb/>
- Saraswat 2021.** Saraswat, Abhishek et al. (2021). Towards energy efficient approx cache coherence protocol. <https://consensus.app/papers/towards-energy-efficient-approx-cache-coherence-protocol-saraswat-abhishek/397cbac8b7175e9795df8538ffe6177/>
- Somarouthu 2025.** Somarouthu (2025). Demystifying cache coherency in modern multiprocessor <https://consensus.app/papers/demystifying-cache-coherency-in-modern-multiprocessor-somarouthu/4c6587e7b010539da5c46fd8cbfa74f2/>
- Tiwari 2014.** Tiwari et al. (2014). Performance comparison of cache coherence protocol on <https://consensus.app/papers/performance-comparison-of-cache-coherence-protocol-on-tiwari/ac6dbbf566dd5e989083148bb990d160/>

Titles ending in an ellipsis were truncated in the source links and are shown as available.